GUI developer guide — ASKET mission cockpit

Who this is for: whoever takes over the mission GUI. You will probably want to change how it looks. That is fine and expected. This document exists so that you can change the design without quietly breaking the boat.

What it assumes: nothing. Sections A to D are for somebody who has never seen this project, and who may be new to web development and has certainly never used ROS. Sections 1 to 7 are the technical reference, for once you are working. Read A to D first even if you are experienced — they take fifteen minutes and they contain the two rules that matter.

If you want to	Read
Understand what this thing is	A. The GUI in 5 minutes
Get it running on your laptop today	B. Your first hour
Look up a word you do not know	C. Glossary
Know what you are allowed to change	D. What you may change, in four lines
Work on the code	1. Architecture onwards

A. The GUI in 5 minutes

What it is

ASKET is a small uncrewed boat. It carries a sonar and drives itself along survey lines over a kelp farm in Namibia, while a person watches from the shore.

This GUI is **the screen that person watches.** It runs in an ordinary web browser on a laptop on the beach, and it shows what the boat is doing right now: where it is, which way it is pointing, how much battery is left, whether the sonar is working, whether the radio link is still good. It also has a handful of buttons: switch the boat between manual and automatic steering, start and stop recording, run the checks before launch.

What it is not

It is not what keeps the boat safe. That is the single most important thing to understand before you change anything.

The person on the beach also holds a radio transmitter, the kind used for model aircraft. One switch on it stops the boat's propellers. That signal goes straight to a small microcontroller on the boat and cuts the motors there, in a few hundred milliseconds, in hardware. It does not travel through the Jetson computer, or through ROS, or through this GUI, or through the browser or the Wi-Fi.

So:

- If your code crashes, the boat is still safe.
- If the laptop's battery dies, the boat is still safe.
- If the Wi-Fi drops, the boat is still safe.
- **And if you add a feature that the boat's safety comes to depend on, you have broken that.** Do not.

The GUI's job is to tell the truth about what the boat is doing. Its failure mode is not crashing — it is lying: showing a position from thirty seconds ago as if it were current, or showing "autonomous" because somebody asked for autonomous rather than because the boat went there.

What is on the screen

Nine panels, roughly: a **map** with the boat, its track, the planned survey lines and the geofence it may not leave; a **pre-flight** panel that runs about twenty checks and gives a GO or NO-GO verdict; a **boat** panel with the mode, the arming state and the mode buttons; **power**; **camera**; **sonar**; **obstacles**; **recording**; and **link**. Across the top is a one-line summary: ready or not ready, and why.

The two rules you will meet everywhere in the code

1. **Show what the boat confirms, never what was asked for.** When you press "Autonomous", the button shows pending until the boat's own status says it is in autonomous mode. If it never does, the GUI says the command failed. It never just colours the button in.

2. **Every number on screen carries its age.** A value that stops updating must visibly go stale, because on a bad radio link a frozen screen that looks live is worse than a blank one.

B. Your first hour

The goal: the whole GUI running on your own laptop, with a simulated boat, in about ten minutes. **You need no boat, no ROS, no Python, no Jetson and no internet beyond the first install.** Everything you will see is generated in the browser.

1. Install Node.js

Node is the runtime that builds and serves the frontend. Get the **LTS** version from https://nodejs.org. On Linux or macOS you may prefer your package manager.

Check it worked — open a terminal and type:

```
node --version
```

You want **v20 or newer**. (Verified here on v22.)

2. Get the code

```
git clone https://github.com/NODE-Engineering-Club/node-ros-namibia.git
cd node-ros-namibia
git checkout integration-pre-sea-trial
```

3. Install the frontend's dependencies

```
cd src/asket_gui
npm install
```

This downloads the libraries into a `node_modules` folder. It takes a minute or two the first time and is the only step that needs the internet. You run it once, and again whenever somebody changes `package.json`.

4. Start it in mock mode

```
npm run dev:mock
```

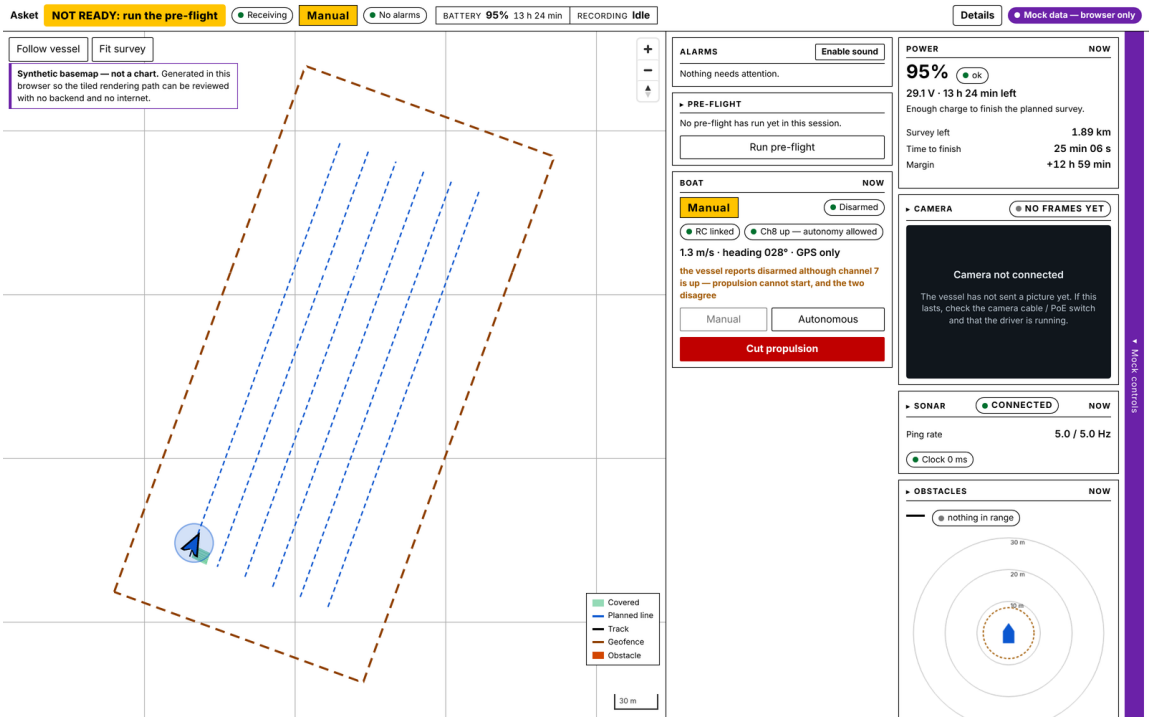
"Mock mode" means the GUI invents its own data instead of talking to a boat. The terminal prints something like:

```
VITE v7.3.6   mock   ready in 1504 ms
➜  Local:   http://localhost:5173/
```

Leave this terminal open. It is the server; closing it stops the GUI.

5. Open it

Put `http://localhost:5173/` in your browser.



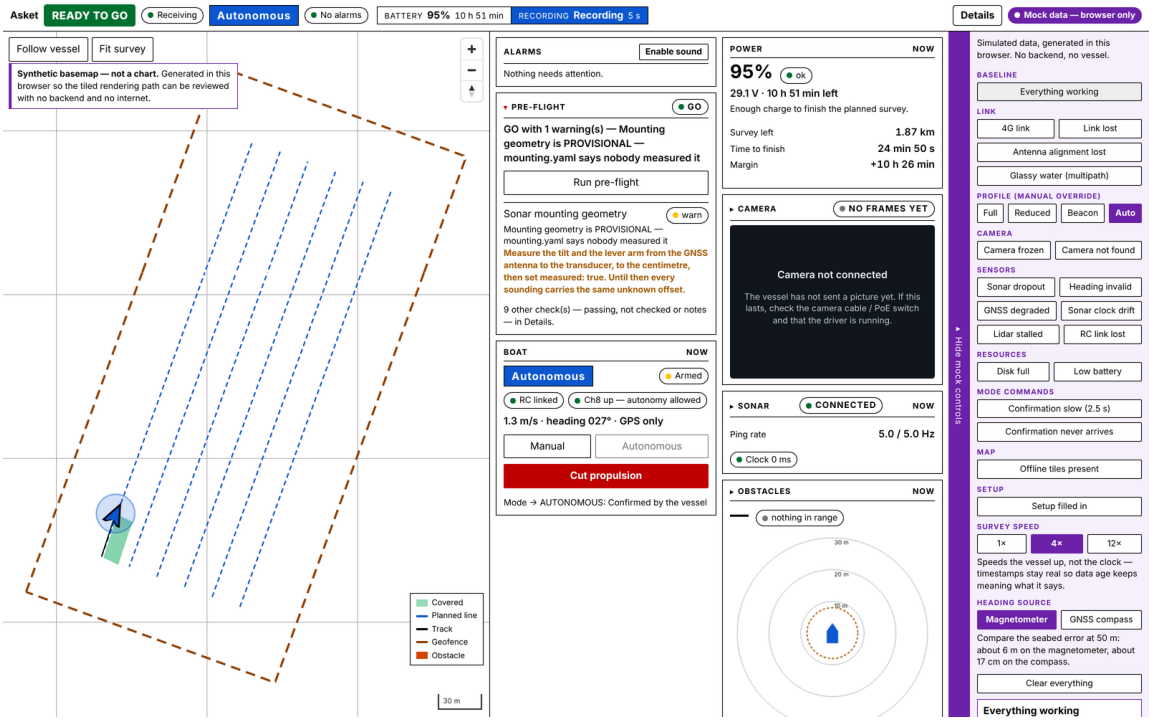
What you should see: the map with a dashed brown geofence and blue planned survey lines, the boat as a small arrow, and panels down the right-hand side. At the top left it says **NOT READY: run the pre-flight**.

Nothing is wrong. A freshly started simulator sits in manual, disarmed and idle. That is deliberately indistinguishable from a boat that is switched off, which is the honest thing for it to be.

6. Press "Everything working"

Look at the right-hand edge of the window for a purple strip labelled **Mock controls**. Click it. A third column slides out — this column exists only in mock mode and is your control over the simulated boat.

Its first button, under **BASELINE**, is **Everything working**. Press it.



What you should see now:

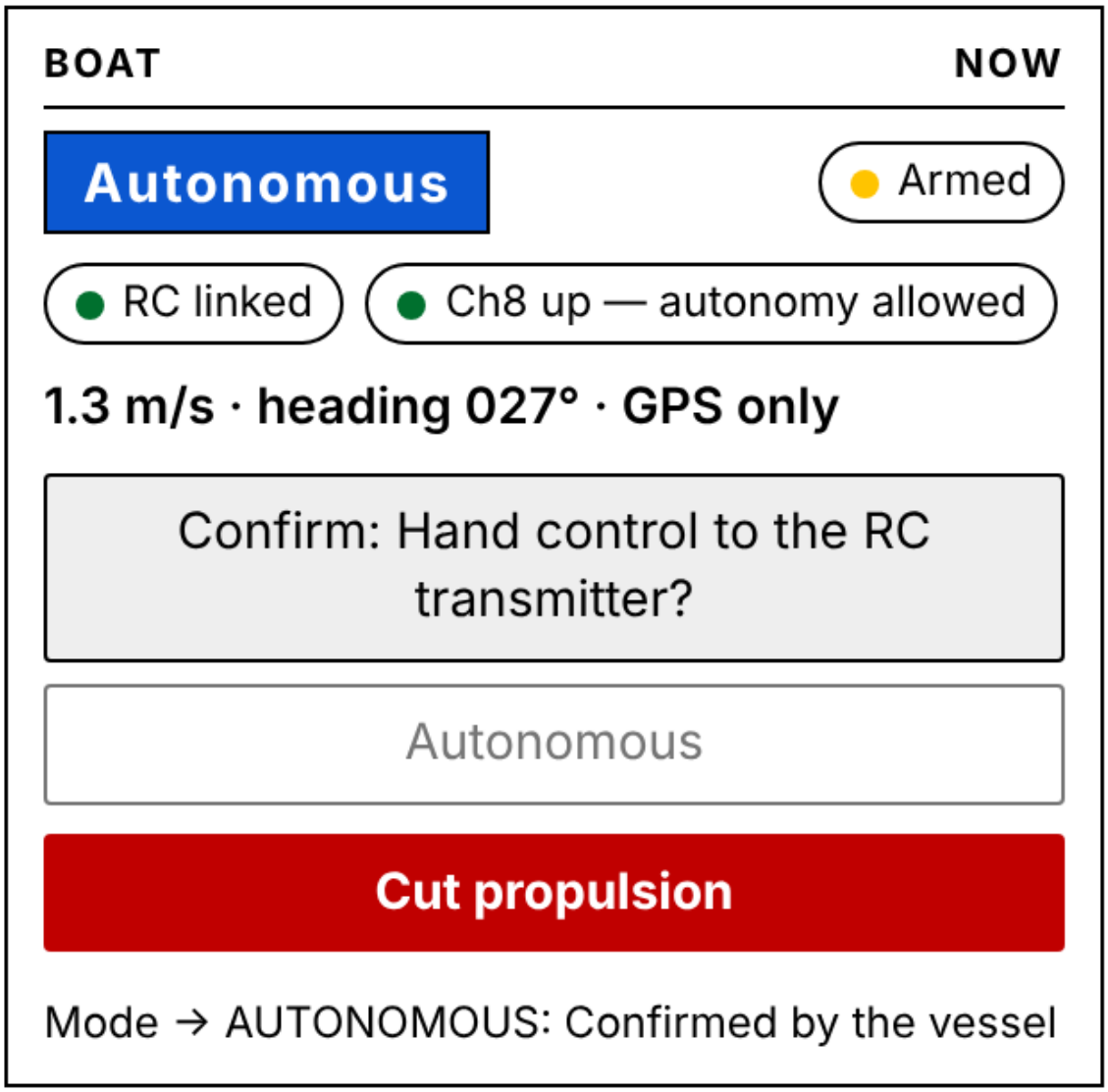
Where	What
Top bar	**READY TO GO** in green, **Autonomous** in blue, battery 95 %, **Recording**
Map	the boat moving along a survey line, painting a pale green covered strip behind it
Pre-flight	a green **GO** badge
Boat	**Autonomous**, **Armed**, "Mode → AUTONOMOUS: Confirmed by the vessel"
Power, Sonar	real-looking numbers; sonar connected at 5 Hz
Camera	"Camera not connected" — correct, there is no camera in the mock

Press this button before anything else, every time. Every other control in that column makes something worse, and you cannot tell whether a degraded state renders correctly until you have seen the healthy one.

Notice the pre-flight says **GO with 1 warning — Mounting geometry is PROVISIONAL**. That warning is real and it is in the simulator on purpose: nobody has measured where the sonar sits on the boat, and the GUI says so rather than hiding it.

7. Watch the two-step confirmation

In the Boat panel, click **Manual** once — just once.



The button does not send anything. It changes into **"Confirm: Hand control to the RC transmitter?"** and waits. Click it again and the command goes; the button reads Waiting… until the simulated boat confirms, and only then does the displayed mode change.

That is safety rules 4 and 5 from section 3, and you have just watched both. If you redesign this panel, this behaviour has to survive.

8. Now break things on purpose

Go back to the mock controls column and work down it. **Link lost**, **GNSS degraded**, **Camera frozen**, **Sonar dropout**, **Low battery**, **RC link lost**, **Confirmation never arrives**. After each one, ask: does the screen tell me the truth about what just happened? Press **Everything working** between experiments to get back to a known state.

Two worth doing slowly:

- **Link lost** — the panels must visibly go stale, not keep showing the last value as if it were current.
- **GNSS degraded, then Run pre-flight, then Autonomous** — the verdict goes NO-GO, and the Autonomous button is pressable but refused, with the reason on screen. Only then does a checkbox and a second button appear to force it. That sequence is explained in section 3; seeing it once is worth more than reading it.

9. When you are ready to change something

`src/asket_gui/src/` holds the code. `panels/` is one file per panel, `components/` the shared pieces, `lib/` the logic. Vite reloads the page as you save. Before you merge anything, work through section 6.

C. Glossary

Words used in this document and in the code, one line each.

Term	Meaning
Backend	The program on the boat's computer that the GUI talks to. Here it is called `gui_backend`. It collects data from the boat and serves it to the browser.
Frontend	The part that runs in the browser: what you see, and what you will be changing.
WebSocket	A connection that stays open so the backend can push updates as they happen, instead of the browser asking over and over. The GUI uses exactly one.
Stream	One named kind of live data, like `vessel` (position and heading) or `power` (battery). The GUI subscribes to the streams it wants; it is sent nothing it did not ask for.
Subscribe	To ask the backend for a stream, at a rate. The backend may grant less than you asked and tells you why.
Link profile	How much data the radio link can carry right now: `full`, `reduced` or `minimal`. Chosen automatically; it decides which streams survive.
Payload	The actual contents of one update on one stream — the JSON object with the numbers in it.
Pre-flight	The boat's self-check: about twenty tests (GNSS, battery, sonar, firmware version, geofence…) run before launch.
GO / NO-GO	The pre-flight's verdict. NO-GO means at least one critical check failed, and the backend will then refuse to put the boat in autonomous mode.
Mock mode	The GUI running with invented data generated in your browser. No boat, no backend, no network. How you develop.
Simulator	The Python version of a fake boat, inside the real backend. One step more realistic than mock mode.
ROS 2	The robotics framework the boat's software is built on. You will not write any. It is where the backend gets real data from.
Topic	A named stream of messages inside ROS 2, such as `/pico/status`. The backend reads topics and turns them into GUI streams.
Pico	The small microcontroller on the boat that drives the motors, reads the radio and owns the safety logic. The "vessel confirms" in rule 1 means the Pico.

Term	Meaning
Killswitch / Ch8	The hardware stop: a physical switch and a channel on the radio transmitter. Both cut the motors without any software involved.
Arming	Whether the motors are allowed to turn at all. Controlled by a switch on the radio transmitter, never by software.
Geofence	A polygon on the map the boat may not leave. Enforced by the navigation stack, not by the GUI.
Stale	A value that has stopped being updated. The GUI must show it as stale rather than as current.

D. What you may change, in four lines

Change freely: every colour, font, icon, animation, panel layout, component structure and build tool. The whole look is yours.

Keep, or get reviewed first: anything that decides what the operator is told — the two-step confirmations, pending-versus-confirmed state, the NO-GO gate and its force path, visible data age, absent hardware showing as absent, and visible link loss.

Never: put the GUI in the safety chain. The radio transmitter and the killswitch stop the boat; nothing you add may weaken that or be relied on instead of it.

When in doubt: the tables in section 3 name the exact mechanism behind each rule, so you can check whether you have broken one.

The technical reference

Everything from here on assumes you have done section B and have the GUI running. It is written for working on the code rather than for a first read.

How facts are labelled in this document

Every technical claim below carries one of these. There is no "should work".

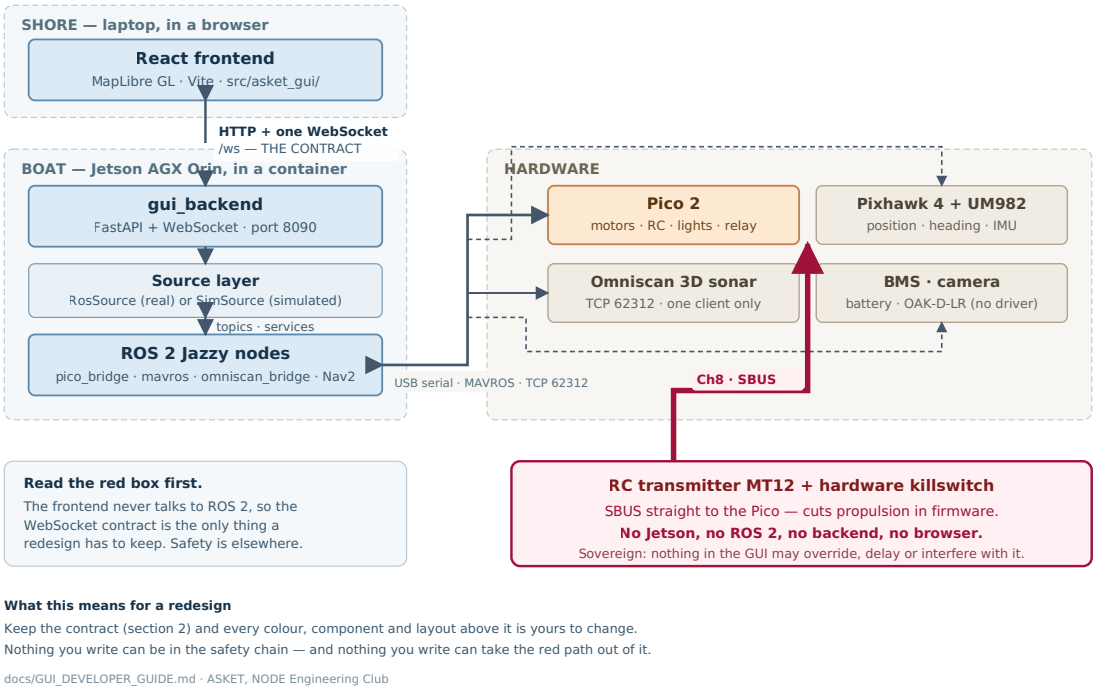
Label	Meaning
VERIFIED	Measured on the real hardware. The date and who did it are given.
TESTED IN SOFTWARE ONLY	Covered by unit tests, the simulator or the browser mock. No hardware involved.
NOT VERIFIED	Written and plausible; nothing has tested it, in software or on hardware.

Label	Meaning
UNKNOWN	Nobody knows. Listed in Open questions.

The habit matters more than the labels. The one failure this GUI is designed against is an operator reading a comfortable number off a screen while the boat does something else.

---

1. Architecture on one page



What this means for a redesign

Keep the contract (section 2) and every colour, component and layout above it is yours to change.
Nothing you write can be in the safety chain — and nothing you write can take the red path out of it.

docs/GUI_DEVELOPER_GUIDE.md · ASKET, NODE Engineering Club

Read the red path first. **The RC transmitter and the hardware killswitch reach the motors without passing through the Jetson, ROS 2, the backend or the browser.** Nothing you write can be in that path, and nothing you write can take it out of that path. That is the whole safety design, and it is why the rest of this document can be relaxed about the GUI crashing.

Layer by layer:

Layer	Where	What it does
Frontend	`src/asket_gui/`	React 19 + MapLibre GL 5, built by Vite. Renders. Holds no authority.
Backend	`src/gui_backend/`	FastAPI. Serves the built frontend, one WebSocket, a few HTTP endpoints. Decides what a client may be sent.
Source	`gui_backend/core/ros_source.py`, `sim_source.py`	One interface, two implementations: the real ROS 2 graph, or the simulator. The hub cannot tell them apart.

Layer	Where	What it does
ROS 2	`src/sensors/`, `src/control/`, `src/omniscan_bridge/`, …	The nodes that actually talk to hardware.

The split that matters for you: **the frontend never talks to ROS 2.** It talks to `gui_backend`, and `gui_backend` talks to ROS 2. So the contract in section 2 is the only thing you have to keep.

---

2. The contract

This section is the load-bearing part of the document. Keep everything here and you can rewrite every component in the frontend.

The protocol's own reference is `docs/ws_protocol.md`. This section says which parts of it are contract rather than description.

2.1 HTTP endpoints

All served by `gui_backend/core/server.py`.

Method and path	Returns	Why it is outside the WebSocket
`GET /`	`index.html` of the built frontend	—
`GET /{any asset}`	static files from the built frontend	—
`GET /api/health`	`{ok, server_utc_ms, clients, profile, source}`	A liveness check that works when the WebSocket does not.
`GET /api/setup`	the setup page's field inventory and current answers	Setup is needed most when the link is worst. A `minimal` link profile could negotiate a stream away; it cannot negotiate this away.
`GET /api/tiles/info`	where the map's tiles come from, and what is missing	The map must be able to say why it is blank.
`GET /tiles/{z}/{x}/{y}.{ext}`	map tiles from the offline `.mbtiles` file	
`GET /tiles/source/{source_id}/{z}/{x}/{y}.{ext}`	map tiles proxied from an online source, cached on the boat	
`GET /tiles/fonts/{fontstack}/{glyph_range}.pbf`	MapLibre glyphs	MapLibre will not render labels without these.
`WebSocket /ws`	everything live	

If the frontend has not been built, `GET /` returns HTTP 503 with `{"error": "frontend not built", "fix": "cd src/asket_gui && npm install && npm run build"}`. That is a real response you will meet; see section 4.

2.2 The WebSocket, in one paragraph

One endpoint, `/ws`, JSON frames both ways, and **the backend pushes nothing until you ask for it**. A client subscribes to named streams at a requested rate and detail; the server decides what it will actually send and always says why it is sending less. This is not politeness — a GUI that subscribes to everything works on a bench and collapses 200 m offshore, and the subscription model is the structural defence against that.

Client to server:

```
{"type": "subscribe",   "streams": [{"name": "vessel", "rate_hz": 5, "detail": "full"}]}
{"type": "unsubscribe", "streams": ["lidar"]}
{"type": "set_profile", "profile": "full"}
{"type": "command",     "id": "c1", "name": "set_mode", "args": {"mode": "MANUAL"}}
{"type": "pong",        "id": 7}
```

Server to client: `hello`, `subscribed`, `data`, `stream_unavailable`, `command_result`, `alarms`, `profile`, `ping`, `error`. Each is defined in `docs/ws_protocol.md`.

Three details you must not drop when you rewrite the client.

1. **Every `data` frame carries two timestamps**, and they mean different things. `source_utc_ms` is when the value was produced by the sensor; `server_utc_ms` is when the frame was sent. Render age from `source_utc_ms`, measured against an estimate of the **server's** clock, not the laptop's. The existing estimator is in `src/asket_gui/src/lib/staleness.js`: skew is `max(server_utc_ms − local receive time)` over a recent window, because transmission delay only ever makes a frame look older, so the maximum is the least-delayed sample. Get this wrong and a laptop with a bad clock renders every value as permanently fresh or permanently hours old, with no way for the operator to tell which. (TESTED IN SOFTWARE ONLY.)
2. **`subscribed` carries a `reason` whenever you got less than you asked for.** Put it on screen. An operator who cannot distinguish "nothing is happening" from "I am not being sent it" will eventually act on the wrong one.
3. **`stream_unavailable` arrives about 5 s after a granted stream has produced nothing.** It means the subscription worked and the data source is silent — a different fault from a refused subscription, and it needs different words.

2.3 Streams

Defined in `gui_backend/core/streams.py`, which is the single source of truth. `critical` streams survive into every link profile, including an LTE-M beacon link.

Stream	Carries	Default / max Hz	Critical
`vessel`	position, heading, course, attitude, GNSS quality	5 / 10	yes
`pico`	confirmed mode, arming, relays, ESCs, RC link	2 / 10	yes
`power`	battery voltage, current, charge, endurance	1 / 2	yes
`alarms`	active alarms	1 / 2	yes
`link`	shore link quality, latency, profile	1 / 2	yes
`heading`	heading provenance, validity, divergence from course	2 / 5	
`mission`	recording state, elapsed time, disk	1 / 2	
`coverage`	swath ribbon painted so far (append-only)	1 / 4	
`track`	vessel track history (append-only)	1 / 4	
`sonar`	sonar health: ping rate, points, packet loss, clock offset	1 / 2	
`lidar`	obstacle returns, top-down	5 / 10	
`diagnostics`	built-in test results and node health	0.2 / 1	
`plan`	survey lines, waypoints, geofence (on change only)	— / 1	
`camera`	forward camera JPEG, with its age on every frame	5 / 15	

`coverage` and `track` send only what is new since you last heard. Do not "simplify" them into sending the whole history: that reached 18 kB per frame after ninety seconds in testing and would have been about a megabyte after three hours.

`camera` is the one stream sent as **binary** rather than JSON, and the one that changes the economics of the link.

2.4 Commands

Defined in `gui_backend/core/commands.py`. A command has **three** outcomes and only three: `pending`, `confirmed`, `failed`.

Command	Does
`set_mode`	request MANUAL / AUTONOMOUS / … (takes `force_nogo` — section 3)
`cut_propulsion`	the soft latch. **Labelled "Cut propulsion", never "Emergency stop".**
`set_ping_parameters`	sonar range, gain, rate
`start_mission`, `stop_mission`	recording
`export_mission`, `delete_mission`, `list_missions`	recordings on disk
`run_system_test`	run the pre-flight
`set_profile`	force a link profile
`save_setup`, `apply_setup`	the setup page. Two commands on purpose: saving is a reversible file write, applying changes how the vessel places every sounding

Command	Does
`inject_fault`, `clear_fault`	simulator only

There is deliberately no fourth outcome in which the UI optimistically shows what was asked for. `confirmed` means the status stream — what the Pico reports the vessel is actually doing — now agrees with the request. It never means "the request was acknowledged". This is safety rule 4 in `docs/safety.md`, and `gui_backend/core/commands.py` enforces it.

2.5 ROS topics and services, for orientation

You will not touch these, but it helps to know the backend is not inventing data. `src/gui_backend/config/topics.yaml` maps each logical source to a topic, a type and optionally an adapter function. **No topic name is hard-coded anywhere in `gui_backend`.** Pointing a stream at a different topic is a change to that file plus, at most, one function in `gui_backend/core/adapters.py`.

Read by the backend: `/gps_driver/gps_raw`, `/odometry/filtered`, `/odometry/gps`, `/mavros/gpsstatus/gps1/raw`, `/imu_driver/imu_raw`, `/autopilot/detected`, `/pico/status`, `/control/effort`, `/sonar/status`, `/mission/state`, `/battery/state`, `/system_test/report`, `/diagnostics`, `/lidar_driver/scan_raw`, `/obstacles/lidar`, `/obstacles/fused`, `/oak/rgb/image_raw/compressed`.

Written by the backend: `/pico/mode_request`, and the services `/omniscan_bridge/set_ping_parameters`, `/omniscan_bridge/reload_mounting`, `/mission_recorder/{start,stop,export,delete}_mission`, `/system_test/run`.

(All 18 topics and 7 services above were checked to exist in the source tree on 2026-11-09. TESTED IN SOFTWARE ONLY — nothing confirms they carry data on the boat; that is Phase P9 of `COMMISSIONING_ROADMAP.md`.)

---

3. What you may change, and what needs a review

Change freely

- Any colour, font, spacing, icon, animation, dark/light theme.
- Panel layout, ordering, grouping, which panels fold, responsive behaviour.

- Component structure, state management, build tooling, the CSS approach.
- Chart and map styling, marker shapes, the basemap's appearance.
- Wording of anything that is **not** in the list below.

Do not change without a review by somebody who knows the hardware

Each of these exists because of a specific way an operator can be misled. The mechanism is named so you can check you have not broken it.

Behaviour	Why	Where it lives
The soft stop is labelled "Cut propulsion", never "Emergency stop"	So no operator comes to rely on a software latch as a real emergency stop. The string "Emergency stop" appears nowhere in the frontend, and that is enforced.	`src/asket_gui/src/lib/labels.js`, command name `cut_propulsion`
Displayed state is confirmed state	The GUI shows what the Pico reports, never what was requested. A pending command must render visibly differently from a confirmed one.	`gui_backend/core/commands.py`, `src/asket_gui/src/panels/ModeCommands.jsx`
Mode commands need two-step confirmation	One misclick must not change the vessel's mode.	`src/asket_gui/src/components/ConfirmButton.jsx`
A NO-GO pre-flight blocks AUTONOMOUS	See below.	`gui_backend/core/hub.py` `_nogo_gate()`, `src/asket_gui/src/panels/ModeCommands.jsx`
The force-past-NO-GO path stays deliberate and logged	See below.	same
Every live value shows its age, and degrades visibly when stale	On an intermittent link an operator must never mistake a stale position for a current one.	`src/asket_gui/src/lib/staleness.js`, `components/Value.jsx`
The killswitch / RC state is displayed truthfully, including "not sent"	"Not wired, so not reported" and "reported as safe" are opposite facts. Never render an absent value as a zero or a good state.	`gui_backend/core/pico_state.py`, `src/asket_gui/src/components/NotFitted.jsx`
Sonar states are not collapsed further	Section 5.	`src/asket_gui/src/panels/SonarPanel.jsx`
Link loss is visible as link loss	A frozen screen that looks live is the worst failure this GUI can have.	`src/asket_gui/src/panels/LinkStatus.jsx`, `lib/connection.js`
Motor tests never run automatically	Two-step confirmation plus an explicit acknowledgement that the vessel is out of the water or moored, and a brief low-power pulse only.	safety rule 6

The NO-GO gate and the force button, in detail

This is the subtlest piece of behaviour in the GUI, so it is worth spelling out.

1. The pre-flight (`system_test`) produces a GO or NO-GO verdict with per-check rows.
2. If the latest verdict is NO-GO and the operator requests AUTONOMOUS, **the backend refuses it** and returns `failed` with `nogo: true` and the reason. The refusal is in the backend, not the button: `hub.py` `_nogo_gate()`.
3. The AUTONOMOUS button is **not** disabled while NO-GO. It is pressed, and it fails with an explanation. This is deliberate — a greyed-out button teaches an operator nothing, and they will not read the pre-flight to find out why.
4. Only after that refusal does the GUI offer a way past: a checkbox ("I have read the pre-flight and accept the NO-GO") plus a second confirm button ("I know it is NO-GO, force"), which sends `set_mode` with `force_nogo: true`.
5. Every forced press is logged by the backend and written to the mission record.

If you redesign the mode panel, keep all five steps. Removing step 3 (by disabling the button) or step 4's two separate actions (by making one click force it) both look like simplifications and both are regressions.

(TESTED IN SOFTWARE ONLY: `src/gui_backend/test/test_nogo_gate.py`.)

And the one rule above all the others

From `docs/safety.md`: **the hardware killswitch and RC channel 8 are sovereign, and the GUI is never in the safety chain.** If the Jetson, ROS 2, the network or the browser fails, safety is unaffected. Do not add a feature whose absence would make the boat less safe — that would put the GUI in the chain, which is the one thing it may not be.

---

4. Running it

Three ways, in increasing order of what has to be installed. Use the lightest one that answers your question.

4.1 Mock mode — Node only, nothing else

Walked through step by step in section B; the short form:

```
cd src/asket_gui
npm install
npm run dev:mock          # prints a http://localhost:5173 URL
```

No ROS 2, no Python, no backend process, no Jetson, no hardware, no internet. Everything on screen is generated in the browser. **This is the mode to develop a design in.** It is a permanent feature of the repository, not scaffolding: two tests keep it honest against the real backend (the negotiation table is generated from `core/streams.py`, and the payload shapes are compared across languages by running the JavaScript under Node).

First thing to press: the dev panel is the third column on the right, and exists only in mock mode. Its first control, under **Baseline**, is **Everything working**. Press it before anything else — a freshly started simulator sits in MANUAL, disarmed and idle, so three panels read "not sent" and the pre-flight says no report exists. Nothing is wrong, but it is indistinguishable from a vessel that is switched off. Every other control on that panel makes something worse, and you cannot judge whether a degraded state reads correctly until you have seen the healthy one.

4.2 Real backend, simulated boat — Node and Python

```
# from the repository root
python3 -m gui_backend.core.app --sim --healthy      # http://localhost:8080
```

The real payload builders, the real WebSocket, the real subscription negotiation; a simulated vessel. Use it when you have changed anything about subscriptions, staleness or commands, because the mock's transport is a stand-in and this one is the real thing.

Needs `pytest`-style access to the GUI packages on `PYTHONPATH`; the repository root `conftest.py` arranges that for tests, and running from the root works.

Useful flags (`gui_backend/core/app.py`): `--port`, `--tiles <file.mbtiles>`, `--static <dir>`, `--time-scale`, `--seed`, `--shape-link`.

`--shape-link` is worth knowing about: it actually throttles the outbound socket to the simulated bearer — serialisation delay from its capacity, half its round-trip time as latency, loss from its quality. Without it, a

"degraded link" in simulation only means fewer streams are subscribed while the wire stays a gigabit loopback, so the thing the whole design exists to survive never gets exercised. (TESTED IN SOFTWARE ONLY.)

4.3 The whole stack, on the boat — needs the workspace built

```
ros2 launch gui_backend gui.launch.py                # the GUI alone, port 8090
ros2 launch bringup njord.launch.py enable_gui:=true # the whole boat
```

Port **8090** on the boat, not 8080. Open `http://<jetson-ip>:8090` from the shore laptop.

For how to get onto the Jetson, where its code actually lives, and the container trap that catches everybody once, read `COMMISSIONING_ROADMAP.md` section 2.

4.4 Building the frontend, and the trap in it

```
cd src/asket_gui
npm install
npm run build
```

Vite writes the output **straight into `src/gui_backend/gui_backend/static/`** (`vite.config.js`, `outDir`), because `gui_backend/setup.py` collects that directory while it runs.

Two consequences that have each cost somebody an afternoon:

1. **`src/gui_backend/gui_backend/static/` is in `.gitignore`.** It is not in the repository. A fresh clone has no frontend, and `colcon build` of `gui_backend` then installs an empty directory — which produces a backend that answers **HTTP 503** with the `npm run build` fix in the body. If you see that 503, this is why. Build the frontend before `colcon build`.
2. **`colcon build --symlink-install` installs the frontend as symlinks.** FastAPI's `StaticFiles` rejects any path resolving outside the directory it was given, so every asset came back 404 while the files were present and readable, and the `/` route still served `index.html` — a blank page with a 200 on it. The fix is `follow_symlink=True` in `server.py`; do not remove it.

The production container image builds the frontend itself, in its own Docker stage, so this only bites in a development checkout.

---

5. The sonar, and why its panel is deliberately dull

The sonar is the part of this GUI most likely to tempt you into adding features. Please read this section before you do.

The Omniscan 3D is operated from SonarView, Cerulean's own application, on its own screen. That is where its range, gain and ping rate are set and where its data is recorded. The mission GUI shows **health only** — whether the sonar is alive — plus a button that opens SonarView.

Which panel you get is config, not code: `sonar_panel: health` in `src/gui_backend/config/topics.yaml` selects the health view; `full` (used by the simulator and the mock) selects the version with controls. Both live in `src/asket_gui/src/panels/SonarPanel.jsx`.

Four states in the diagnostics, three words in the GUI

`omniscan_bridge` publishes four distinguishable conditions on `/diagnostics`:

`/diagnostics` keys	Condition	What an operator should do
`link_up: False`	no TCP connection	wrong address, or nothing listening — read `last_connect_error`
`link_up: True`, `bytes_received: 0`	connected, sonar sending nothing	dry transducer, or the stream has to be asked for
`bytes_received > 0`, `point_sets: 0`	something arriving, and it is not what we parse	capture it with `tcpdump`
`point_sets > 0`	working	—

The GUI's health panel collapses these into **`sonar alive`** (connected and fresher than 10 s), **`sonar silent`** (anything else), or **nothing at all** when `omniscan_bridge` is not running, where it says so explicitly rather than showing a reassuring blank.

This is a known limitation, not a design goal. `sonar silent` sends an operator to check a cable when the real cause may be a dry transducer or an unparsed stream. **If you improve one thing about the sonar in this GUI, make it this one** — the data is already on `/diagnostics`; it is the display that is coarse. What you must not do is collapse it further, or render the absent case as a healthy one.

Three hardware facts that constrain the UI

1. **Only one TCP client at a time.** VERIFIED 2026-11-06 (Salvador Cantuarias Brañes, bench): with one connection to port 62312

open, a second is refused; with the first closed, the port opens again. So **SonarView or `omniscan_bridge`, not both.** Tested with plain TCP clients, not with SonarView itself — whether SonarView behaves the same is UNKNOWN. If the GUI ever offers to "connect to the sonar", it has to account for SonarView holding the port, and vice versa.

2. **The sonar keeps pinging when nothing is connected to it.** VERIFIED 2026-11-06: the ping counter ran at 5.0 Hz throughout about 14 minutes that included the bridge being stopped. **Closing the GUI does not stop the sonar.** Stopping it is `ros2 service call /omniscan_bridge/stop_pinging std_srvs/srv/Trigger`, and it must be done before the transducer leaves the water, because pinging in air can damage it. If you put a sonar control in the GUI, this is the one that matters.
3. **The bridge never starts the sonar pinging on its own.** Its `send_ping_parameters` parameter is `false` by default, deliberately: those parameters start the transducer, and a launch file runs when nobody is beside the boat.

---

6. Non-regression checklist before merging a design change

Run these. The first three are cheap and catch most things.

Automated

```
# from the repository root — no ROS 2 needed
pytest src/gui_backend                                  # backend + contract tests
python3 -m flake8 --max-line-length=100 src/gui_backend

cd src/asket_gui
npm install            # needed once, and needed by the tests below
npm run build          # must succeed, and must write into gui_backend's static/
```

`pytest src/gui_backend` includes the tests that exist specifically to stop this GUI drifting away from the hardware. The ones to care about:

Test	Stops
`test_nogo_gate.py`	AUTONOMOUS being reachable over a NO-GO verdict without the force path
`test_commands.py`, `test_mission_and_confirmations.py`	a command showing `confirmed` on acknowledgement rather than on observed state
`test_gui_readiness.py`	the readiness line claiming more than the data supports
`test_fitted_sensors.py`	absent hardware rendering as working hardware
`test_mock_payload_shapes.py`	the browser mock drifting from the real payloads

Test	Stops
`test_mock_policy.py`, `test_mock_link_budget_matches.py`	the mock's negotiation drifting from `core/streams.py`
`test_degraded_link.py`, `test_link_profile.py`	a profile change not actually reducing what is sent
`test_panels_render.py`	a panel throwing on a payload it will really receive
`test_gui_alarm_sound.py`	amber alarms making noise, red ones staying silent

Note: `test_gui_alarm_sound.py` runs the frontend's JavaScript under Node, so it **errors with `ERR_MODULE_NOT_FOUND: react` until you have run `npm install` in `src/asket_gui`**. That is an environment issue, not a failure of your change.

By hand, in mock mode

```
cd src/asket_gui && npm run dev:mock
```

1. Press **Everything working**. Every panel populated, no "not sent", GO pre-flight.
2. Check a stale value: in the dev panel, stop the vessel stream or inject a link fault. Values must **visibly degrade** and show their age. A number that keeps looking live is a bug, however pretty.
3. Request AUTONOMOUS while the pre-flight is NO-GO. It must be **pressable**, must **fail** with the reason on screen, and only then offer the checkbox and the force button.
4. Send a mode command. It must render as **pending**, distinguishably, before it becomes confirmed.
5. Confirm the soft stop still reads **"Cut propulsion"**. If the words "Emergency stop" are anywhere on screen, stop and fix it.
6. Degrade the link to `reduced`, then `minimal`. The GUI must say which profile it is on, and say what it is no longer receiving — not go quietly blank.
7. Disconnect the backend (stop the process, or pull the network). The GUI must say it has lost the link, not freeze on the last frame.
8. Check the sonar panel with `omniscan_bridge` absent: it must say health is not monitored, not show a healthy sonar.
9. Resize to a phone width. The status strip and the mode panel must stay usable — an operator on a beach may be on a tablet.

Ask yourself

- Did I change anything in the "needs a review" table in section 3? If yes, get it reviewed by somebody who knows the boat.
- Does any new value on screen carry its age?

- Does any new state render absence as a good value?

---

7. Known limitations

Limitation	Label
The sonar panel shows three words for four distinguishable states (section 5)	VERIFIED behaviour of the current code
The whole GUI has never been run against the real boat end to end	NOT VERIFIED — Phase P9 of the commissioning roadmap
`/oak/rgb/image_raw/compressed` is the camera topic the GUI expects; the OAK-D-LR driver is not installed and the topic name has not been confirmed on hardware	UNKNOWN
The lidar panel exists; no lidar is fitted to the boat, so it reads "not fitted"	VERIFIED by the configuration
Obstacle data: there is **no obstacle detection on this boat at all**. The GUI cannot show what the boat cannot sense	VERIFIED by `docs/HARDWARE_DECISIONS.md` §5
RTK fixed / float needs corrections reaching the GNSS; not implemented, so the best the GUI can show is "GPS only"	VERIFIED by the configuration
Map tiles need either an `.mbtiles` file on the boat or internet at the shore station	TESTED IN SOFTWARE ONLY
The Jetson's own browser has no WebGL in some configurations, so MapLibre will not render there. Use the shore laptop	NOT VERIFIED on this Jetson — reported during earlier work
Camera stream cost on the Jetson (JPEG encode plus the vision node) has never been measured, which is why the ceiling is 15 Hz and not 30	UNKNOWN

Open questions

Carried from the repository's own registers (`docs/HARDWARE_DECISIONS.md` §6 and `docs/SONAR_BRINGUP.md`), limited to the ones that could change the GUI:

1. **Does SonarView hold the sonar's TCP port the way a plain client does?** UNKNOWN. If it does, the GUI can never show live sonar health while SonarView is recording, and the panel should say so explicitly.
2. **What is the OAK-D-LR's real topic name?** UNKNOWN until the driver is installed. The camera panel is written against `/oak/rgb/image_raw/compressed`.
3. **Which subnet does the sonar end up on?** UNKNOWN — a network decision nobody has taken. The plan is `10.10.20.0/24`, the sonar answers on `192.168.2.21`, and `.21` in the plan belongs to the OAK-D-LR. The GUI is not affected directly; the Open SonarView address on the Setup page is.

4. **Does the sonar's `pt_type` ever report bottom?** UNKNOWN. Everything in the bucket test was unclassified, so the boat's seabed-only filter would publish nothing. If it stays that way on real water, the GUI's coverage ribbon has nothing to paint.

Where to look next

For	Read
The protocol in full	`docs/ws_protocol.md`
The safety rules and how the code enforces them	`docs/safety.md`
The boat's hardware, addresses and what is verified	`docs/HARDWARE_DECISIONS.md`
Mock mode in detail	`src/asket_gui/README.md`
The setup page's design	`docs/setup_page.md`
Map tiles	`docs/map_tiles.md`
Getting onto the Jetson, containers, ROS basics, testing the hardware	`docs/COMMISSIONING_ROADMAP.md`
What the operator is told to do	`docs/OPERATOR_CHECKLIST.md`